

Artificial Intelligence Exam Notes

CT 710 (BEI, IV-I) · CT 653 (BCT, III-II) · CT 78506 (BEX, IV-II) · TU, IOE · tiers over 41 papers, 2069–2082 BS

4. Knowledge Representation, Inference and Reasoning

4 hrs · asked in **all 41 papers** · 4 syllabus sub-topics · 638 marks, **18.8 % of every mark ever set**, against a 7.8 % share of the teaching hours · 33 resolution proofs and 15 Bayes numericals worked in Ch4 – *Knowledge Representation, Inference and Reasoning – Solved Problems*.

Read this first:

- **This is the highest-paying chapter in the subject, by a factor of 2.4.** Four teaching hours, nearly a fifth of every mark. Whatever time the timetable suggests for it, spend more.
 - **One question dominates everything: prove a goal from given premises by resolution refutation** **TOP 33/41**. It is in **33 of the 41 papers**, always for 6 to 10 marks. The premises change; the procedure never does. Learn the procedure once and the question is answered for the rest of your life.
 - Its two feeder topics are asked in their own right: **conversion to CNF** **TOP 9/41** and the **rules of inference**. You cannot resolve without CNF, so this is one topic wearing two hats.
 - The other two reliable topics: **forward Vs backward chaining** **TOP 11/41**, always a comparison with an example, and **belief / causal networks** **HOT 7/41**.
 - **Fifteen papers set a Bayes numerical**, usually bolted onto a 2–3 mark theory opener. The arithmetic is one formula; all 15 are worked in the companion.
 - Pairs the examiner reliably asks together: a resolution proof preceded by **the CNF steps, skolemization or the rules of inference**, and a Bayes numerical preceded by **prior/posterior**. The short part is always the definition; the long part is always the work.
 - **The 5-mark saver:** if a resolution proof stalls in the exam, write the FOPL, write the clause form, write the negated goal, and show the resolution steps you do have. The marks are spread across those stages, not parked on the empty clause.
-

4.1 Knowledge, representation and reasoning

What is knowledge, representation and reasoning? What kinds of knowledge are there? 2 2+5

72 Ash 2+5, 77 Ch 2, 72 Ma 7 · never asked alone: it is the opening 2 marks of a chaining or rule-based-reasoning question, so **answer it in four lines and move on**

The three words

- **Knowledge** = facts, concepts, rules and relationships about a domain that an agent can use. It sits above data and information:
 - **data** → raw symbols; **information** → data with context; **knowledge** → information plus how to use it; **intelligence** → knowledge applied to a new situation.
- **Representation** = writing that knowledge in a formal notation a machine can store and manipulate. Not any notation: one that **supports inference**.
- **Reasoning** = deriving new sentences from the stored ones, so the agent knows things nobody explicitly told it.
- The four questions KR exists to answer: how do we represent facts about the world, how do we reason about them, which representation suits the real world, and how do we put it in a form a machine can compute with.

What a good representation must give you

- **Representational adequacy** – can it express everything in the domain?
- **Inferential adequacy** – can new knowledge be derived from it?
- **Inferential efficiency** – can the derivation be steered, so the search does not explode?
- **Acquisitional efficiency** – can new knowledge be added easily?

The five kinds of knowledge

Kind	What it is	Example
Declarative	Knowing <i>that</i> . Facts, concepts, objects, stated without saying how to use them	"Kathmandu is the capital of Nepal"
Procedural	Knowing <i>how</i> . Rules, strategies, procedures, agendas	the steps to sort a list
Heuristic	Rules of thumb from experience: good, not guaranteed	in chess, keep the opponent's king exposed
Structural	Relationships between concepts and objects	"a cow <i>is-a</i> mammal"
Meta	Knowledge <i>about</i> knowledge: what the agent knows, and how reliable it is	"my map of Pokhara is out of date"

- **Name all five with one example each.** Papers award a mark per kind, and *meta-knowledge* and *structural* are the two students forget.

Why logic is the representation of choice

- It is concise, unambiguous, context-insensitive and expressive, and it comes with a **proof theory**, i.e. **inference is defined for you**. That last property is what the rest of this chapter is built on.
- Logic is fixed by three things, and papers ask for them by name: **syntax** (which strings are sentences), **semantics** (what a sentence means about the world) and **proof theory** (which new sentences may be derived from old ones).

What is a knowledge-based agent? How does it work? 2+3+2

PIN 1/41 82 Ba 2+3+2 · no lecture deck in the set covers this by name verified/added · answer it as the *three operations* plus the two levels

Definition and structure

- An agent whose behaviour comes from a **knowledge base** (KB) of sentences about the world, plus an **inference engine** that derives new sentences from it. It **acts on what it can prove**, not on hard-wired reflexes.
- Two levels, and papers like the distinction:
 - **Knowledge level** – *what* the agent knows and what goal it has, independent of any notation.
 - **Implementation level** – the sentences, the data structures and the algorithm that actually run.
- Three operations, in this order, once per time step:
 - **TELL** the KB what was perceived;
 - **ASK** the KB which action to take;
 - **TELL** the KB that the action was taken.

Why it matters, with the standard example

- The **Wumpus world**: an agent in a cave perceives *breeze*, *stench*, *glitter*. No single percept names a pit, but **breeze in (1,2) and (2,1) with none in (1,1)** proves where the pit is. That inference, not the percept, is the whole point.
- ✓ It can be **told new facts** without reprogramming: add a sentence and the behaviour changes.
- ✓ It handles **partially observable** worlds, because it reasons about what it has not seen.
- × It is only as good as its KB, and inference costs time: a large KB with an undirected engine is exactly the search-explosion problem of Ch 3.
- **Every expert system** (Ch 7) is a knowledge-based agent with a domain KB and a chaining inference engine. Say that sentence and the examiner sees you have connected the chapters.

4.2 Propositional logic: WFF, tautology and the rules of inference

Define a WFF with its properties. Prove that $(A \wedge (A \rightarrow B)) \rightarrow B$ is a tautology 4+4

PIN 1/41 71 Bh 4+4 · the proof is a **4-row truth table**, worked in the companion · “short note: predicate logic / sentence validity” 75 Bh 4, 72 Ma 4 wants the same vocabulary

Proposition and well-formed formula

- A **proposition** is a declarative statement that is **either true or false, never both**. “5 is a prime number” is one; “close the door” is not.
- An **atomic** sentence is a single propositional symbol (P , Q). A **compound** sentence joins atomic ones with the connectives $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$.
- A **WFF** is a string built only by these recursive rules:
 - any propositional symbol is a WFF;
 - if α is a WFF then $\neg\alpha$ and (α) are WFFs;
 - if α, β are WFFs then so are $(\alpha \wedge \beta)$, $(\alpha \vee \beta)$, $(\alpha \rightarrow \beta)$, $(\alpha \leftrightarrow \beta)$;
 - **nothing else is a WFF**. That closing clause is worth a mark and is the one students leave out.
- Precedence, when brackets are missing: $\neg$, then $\wedge$, then $\vee$, then $\rightarrow$, then $\leftrightarrow$.

The four properties to name

- **Valid** (a **tautology**): true under every interpretation. Eg $P \vee \neg P$.
- **Satisfiable**: true under *at least one*. Eg $P \wedge Q$.
- **Unsatisfiable** (a **contradiction**): true under *none*. Eg $P \wedge \neg P$.
- **Equivalent**: $\alpha \equiv \beta$ when they have the same truth value in every row. Equivalently, $\alpha \leftrightarrow \beta$ is valid.
- **The link resolution depends on**: α is valid iff $\neg\alpha$ is unsatisfiable. That single line is why proving a goal is done by **refuting its negation** in §4.5.

Equivalences you must be able to quote

- $\alpha \rightarrow \beta \equiv \neg\alpha \vee \beta$ (**the workhorse**)
- $\alpha \leftrightarrow \beta \equiv (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$
- De Morgan: $\neg(\alpha \wedge \beta) \equiv \neg\alpha \vee \neg\beta$; $\neg(\alpha \vee \beta) \equiv \neg\alpha \wedge \neg\beta$
- Distribution: $\alpha \vee (\beta \wedge \gamma) \equiv (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$
- **Contrapositive** $\neg\beta \rightarrow \neg\alpha$ is equivalent to $\alpha \rightarrow \beta$; the converse $\beta \rightarrow \alpha$ and the inverse $\neg\alpha \rightarrow \neg\beta$ are **not**.

List the rules of inference. When can we use them? 2 3 2+3

78 Ch 3, 80 Ash 3, 81 Ch 2+3, 76 Ba 2, 70 Ma 3 · 5 papers, none over 3 marks on its own, so no tier chip · it is always the opener to a resolution proof, which is where the other 7 or 8 marks are

The eight propositional rules

Rule	From $\therefore$ conclude	Tautological form
Modus ponens	$P \rightarrow Q, P \therefore Q$	$[(P \rightarrow Q) \wedge P] \rightarrow Q$
Modus tollens	$P \rightarrow Q, \neg Q \therefore \neg P$	$[(P \rightarrow Q) \wedge \neg Q] \rightarrow \neg P$
Hypothetical syllogism	$P \rightarrow Q, Q \rightarrow R \therefore P \rightarrow R$	chains two implications
Disjunctive syllogism	$P \vee Q, \neg P \therefore Q$	eliminates a disjunct
Addition	$P \therefore P \vee Q$	weakens
Simplification	$P \wedge Q \therefore P$	splits
Conjunction	$P, Q \therefore P \wedge Q$	joins
Resolution	$P \vee Q, \neg P \vee R \therefore Q \vee R$	the only one §4.5 needs

The four quantifier rules

- **Universal instantiation:** from $\forall x P(x)$ infer $P(c)$ for any constant c .
- **Existential instantiation:** from $\exists x P(x)$ infer $P(c)$ for a **brand-new** constant c . Reusing an existing name is the classic mistake; this is skolemization in one line.
- **Universal generalisation:** from $P(c)$ for an arbitrary c infer $\forall x P(x)$.
- **Existential generalisation:** from $P(c)$ infer $\exists x P(x)$.

When can we use them (the second half of the question)

- Only when the premises **match the pattern exactly**, after unification. A rule is a *syntactic* licence: it never looks at meaning.
- They are **sound**: applied to true premises they can only produce true conclusions, so a derivation is a proof.
- They are used when the KB is in the right form: modus ponens needs implications, **resolution needs CNF** (§4.4), and forward/backward chaining (§4.6) need *definite* clauses.

4.3 First-order predicate logic

Why do we need FOPL? Explain its syntax and quantifiers 2 2+3 4

70 Ma 2+3, 75 Bh 4, 72 Ma 4 · translating English into FOPL is the first step of every resolution proof, so this section is really 33 papers wide · a translation set is worked in the companion (76 Bh 2)

Why propositional logic is not enough

- In PL a whole statement is **one indivisible symbol**. “All humans are intelligent” and “Some humans are intelligent” become two unrelated letters P and Q , with no way to see that one implies the other.
- It cannot express **objects, their properties or relations between them**, and it has **no quantifiers**, so a fact about a million individuals needs a million sentences.
- FOPL fixes exactly that: it breaks a sentence into **subject** and **predicate**, and quantifies over objects.
- The three-line answer: PL has **limited expressive power**; FOPL adds objects, predicates and quantifiers; so one FOPL sentence replaces an unbounded set of PL ones.

Syntax: the seven elements

- **Constants** – *John, Nepal*, 2. Named objects.
- **Variables** – x, y, z .
- **Predicates** – *Brother, Loves, >*. Relations, true or false of their arguments.
- **Functions** – *FatherOf, sqrt*. Return an *object*, not a truth value. **Predicate Vs function is a favourite one-mark trap.**
- **Connectives** $\neg, \wedge, \vee, \rightarrow, \leftrightarrow$, **equality** $=$, and **quantifiers** $\forall, \exists$.

Skolemization (81 Ba 2, 69 Bh, short note)

- **Definition:** removing an existential quantifier by replacing its variable with a *new* symbol, so the sentence can be written as a clause.
- **Two cases, and the second is the one papers test:**
 - $\exists$ **not inside any** $\forall \rightarrow$ replace with a new **Skolem constant**. $\exists x \text{ rich}(x) \Rightarrow \text{rich}(A)$.
 - $\exists$ **inside** $\forall \rightarrow$ replace with a **Skolem function** of the enclosing universal variables. $\forall x \exists y \text{ loves}(x, y) \Rightarrow \text{loves}(x, F(x))$.
- **Why a function and not a constant:** the person loved depends on who is loving. A constant would assert that *everyone loves the same individual*, which is a different and stronger claim, and the proof built on it is wrong.
- The new symbol must be one **not already used** anywhere in the KB.

The two quantifiers, and the connective each takes

- $\forall$ (**universal**, “for all”) goes with $\rightarrow$:
 $\forall x \text{ man}(x) \rightarrow \text{drinks}(x, \text{coffee})$
- $\exists$ (**existential**, “there exists”) goes with $\wedge$:
 $\exists x \text{ student}(x) \wedge \text{fails}(x, \text{maths})$
- **Swap the connectives and the sentence is nonsense.**
 $\forall x (\text{man}(x) \wedge \text{drinks}(x, \text{coffee}))$ says *everything in the universe is a coffee-drinking man*;
 $\exists x (\text{student}(x) \rightarrow \text{fails}(x))$ is satisfied by any non-student. This single pairing earns or loses the translation marks.
- **Duality**, needed in every CNF conversion:
 $\neg \forall x P \equiv \exists x \neg P$ and $\neg \exists x P \equiv \forall x \neg P$.

Translation patterns worth memorising

- “All A are B” $\rightarrow \forall x A(x) \rightarrow B(x)$
- “Some A are B” $\rightarrow \exists x A(x) \wedge B(x)$
- “No A is B” $\rightarrow \forall x A(x) \rightarrow \neg B(x)$, or $\neg \exists x (A(x) \wedge B(x))$
- “A has at least one B” $\rightarrow \exists x B(x) \wedge \text{has}(A, x)$
- “Everyone who ... all animals” $\rightarrow \forall$ *inside* the antecedent of another $\rightarrow$; that nesting is what makes the standard CNF example in §4.4 hard.

Horn clause and definite clause (78 Ch 3, 70 Ma)

- A **clause** is a disjunction of literals: $\neg P \vee Q \vee \neg R$.
- **Definite clause:** a clause with **exactly one positive** literal. $\neg P \vee \neg Q \vee K$.
- **Horn clause:** a clause with **at most one positive** literal. So every definite clause is a Horn clause, and the extra case is the **goal clause** with no positive literal at all.
- **Why the restriction exists**, which is what the 3 marks are for: a definite clause is exactly an implication with a single conclusion, $(P \wedge Q) \rightarrow K$, so
 - inference collapses to **modus ponens**, and
 - **forward and backward chaining** (§4.6) become possible, in time *linear* in the size of the KB rather than exponential.
- **Prolog** is exactly this: a program is a set of definite clauses.

Unification and the most general unifier 2 3

asked inside resolution questions, never alone · **but no resolution step is legal without it**, and examiners take marks off a proof that unifies wrongly

What it does

- **Unification** finds a substitution θ that makes two atomic sentences identical: $\text{UNIFY}(\alpha, \beta) = \theta$ where $\alpha\theta = \beta\theta$.
- The substitution with the fewest commitments is the **most general unifier (MGU)**, and it is the one to use: a more specific one can lose proofs.
- Written $[\text{term}/\text{variable}]$, Eg $\theta = [\text{John}/x]$.
- **It fails**, and the resolution step is then illegal, when
 - the predicate symbols or the arities differ;
 - two different constants must match, Eg $\text{King}(\text{John})$ Vs $\text{King}(\text{Ram})$;
 - a variable would have to be bound to a term containing itself (the *occurs check*).

Worked, in the form the papers expect

- $\text{King}(x)$ and $\text{King}(\text{John}) \rightarrow \theta = [\text{John}/x]$.
- $P(x, y)$ and $P(a, f(z)) \rightarrow \theta = [a/x, f(z)/y]$.
- $P(x, f(y))$ and $P(a, f(g(x))) \rightarrow \theta = [a/x, g(a)/y]$: substitute a for x first, then **apply that binding inside the second term too**. Writing $g(x)/y$ and stopping is the common half-answer.
- $Q(a, g(x, a), f(y))$ and $Q(a, g(f(b), a), x) \rightarrow \theta = [f(b)/x, b/y]$.
- **Standardise apart first**: if both clauses use x , rename one of them. Two clauses that share a variable name will unify wrongly and the proof will be rejected.

4.4 Conversion to conjunctive normal form

Why is CNF required? Explain all the steps involved in converting to CNF, with an example 2+6 3 5

8 10

TOP 9/41 69 Po 10, 70 Bh 8, 75 Ba 2+6, 79 Ch 2+6, 75 Bh 2+6, 76 Bh 5, 80 Ch 3, 81 Ch 2+3, 76 Ba 4 · **four more papers** give a formula and ask for its CNF **HOT 4/41** 73 Bh 2+2+2+3, 74 Ma 6, 72 Ash 8, 79 Jth 2+2+4, all worked in the companion

Why CNF is required (the 2 marks, and the only part students skip)

- **CNF** = a conjunction of clauses, each clause a disjunction of literals: $(A \vee \neg B) \wedge (C \vee D \vee \neg E)$
- **Resolution is a single inference rule and it only operates on clauses**. Everything else – implications, biconditionals, nested negations, quantifiers – has to go before the machinery can start.
- Because the whole formula is one big **AND**, each clause can be **asserted separately** and stored as its own line in the KB.
- **Every** sentence of propositional or first-order logic has a CNF equivalent, so nothing is lost by insisting on it. That completeness is what makes resolution a *general* proof procedure.
- Contrast **DNF**, a disjunction of conjunctions. Papers sometimes ask for both: CNF is joined by AND, DNF by OR.

The eight steps, in this order

- 1. Eliminate $\leftrightarrow$** : $\alpha \leftrightarrow \beta \Rightarrow (\alpha \rightarrow \beta) \wedge (\beta \rightarrow \alpha)$.
- 2. Eliminate $\rightarrow$** : $\alpha \rightarrow \beta \Rightarrow \neg \alpha \vee \beta$.
- 3. Move $\neg$ inwards**, until it sits on atoms only: De Morgan, $\neg \neg \alpha \equiv \alpha$, and the quantifier duals $\neg \forall x P \equiv \exists x \neg P$, $\neg \exists x P \equiv \forall x \neg P$.
- 4. Standardise variables apart**: rename so that no two quantifiers bind the same variable name.
- 5. Skolemize**: drop each $\exists$, replacing its variable by a new constant, or by a **function** of the enclosing universal variables (§4.3).
- 6. Drop the universal quantifiers**: every remaining variable is understood to be universally quantified.
- 7. Distribute $\vee$ over $\wedge$** : $\alpha \vee (\beta \wedge \gamma) \Rightarrow (\alpha \vee \beta) \wedge (\alpha \vee \gamma)$.
- 8. Split the conjunction into separate clauses**, and rename variables again so no two clauses share one.

Steps 4, 7 and 8 are the ones the local textbook leaves out, and a proof that skips 4 or 8 unifies two unrelated variables and derives nonsense. Write all eight; the marks are per step.

The worked example the papers use (79 Ch, 75 Bh, 73 Bh)

- “Everyone who loves all animals is loved by someone.”
- **FOPL:**
 $\forall x [\forall y \text{ Animal}(y) \rightarrow \text{Loves}(x, y)] \rightarrow [\exists z \text{ Loves}(z, x)]$
- **2. Eliminate $\rightarrow$, twice:**
 $\forall x \neg[\forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)] \vee [\exists z \text{ Loves}(z, x)]$
- **3. Move $\neg$ in, using the dual and De Morgan:**
 $\forall x [\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists z \text{ Loves}(z, x)]$
- **5. Skolemize.** Both $\exists$ are inside $\forall x$, so both become **functions of x :** $y \Rightarrow F(x)$, $z \Rightarrow G(x)$:
 $\forall x [\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))] \vee \text{Loves}(G(x), x)$
- **6. Drop $\forall$, 7. distribute, 8. split:**
 - $\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)$
 - $\neg \text{Loves}(x, F(x)) \vee \text{Loves}(G(x), x)$
- **Read the Skolem functions back:** $F(x)$ is “the animal x does not love, if there is one” and $G(x)$ is “someone who loves x ”. Saying that in one line shows the step was understood, not copied.

Where marks are actually lost

- **Skolemizing to a constant inside a $\forall$.** $\forall x \exists y$ becomes $F(x)$, never A . This is the single commonest error, and it silently changes the meaning.
- **Distributing where you must not.** $\neg(A \wedge \neg B) \vee C$ is $\neg A \vee B \vee C$ – one clause. It is **not** $(\neg A \vee C) \wedge (B \vee C)$. **The local textbook makes exactly this error** in its “likes all food” problem, then quietly uses the correct single clause in the proof underneath. The companion works that family properly.
- **Forgetting to standardise apart.** Two clauses both using x will unify on a variable they do not share in meaning.
- **Dropping $\forall$ before skolemizing.** The order in the list is not decoration: once the $\forall$ is gone there is nothing to make the Skolem function depend on.
- **Simplification is allowed and is worth doing:** a clause containing both P and $\neg P$ is a tautology and can be dropped, and duplicate literals in one clause collapse.

4.5 Resolution refutation**Prove the given goal from the given premises using resolution refutation** 6 7 8 9 10

TOP 33/41 69 Bh 10, 75 Ba 8, 77 Ch 8, 82 Ka 8, 81 Ash 2+6, 81 Bh 9, 82 Bh 6, 70 Bh 8, 71 Bh 8, 72 Ash 7, 74 Bh 8, 73 Bh 8, 71 Bh 8, 80 Bh 8, 80 Ba 8, 72 Ma 6+4, 72 Ma 7, 73 Bh 8, 78 Po 8, 74 Bh 8, 79 Jth 6, 79 Bh 8, 78 Ba 8, 73 Ma 10, 81 Ba 2+6, 81 Ch 8, 78 Ch 3+3, 80 Ch 3+5, 80 Ash 3+5, 79 Ash 7, 76 Ba 2+8, 76 Bh 7, 77 Ch 8 · **33 of 41 papers, 260 marks** · every one of them is worked in the companion

The procedure. This is the chapter.

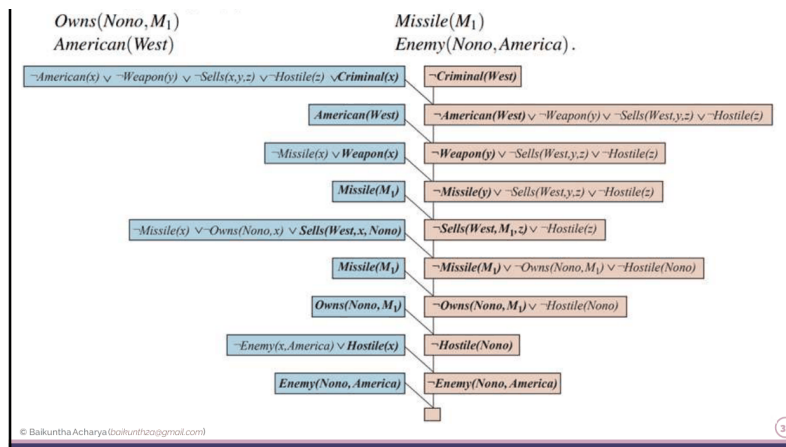
- **1. Convert every premise into FOPL.**
- **2. Convert every FOPL sentence into CNF (§4.4), and number the clauses.**
- **3. Negate the goal,** convert that to CNF too, and add it to the clause set.
- **4. Repeatedly resolve** two clauses containing complementary literals, unifying them with the MGU, and add the **resolvent**.
- **5. Stop when the empty clause $\square$ is derived.** The clause set is then unsatisfiable, so the negated goal is false, so **the goal is true**.

Why refute instead of prove directly

- Resolution is **refutation complete**, not complete: it is guaranteed to find a contradiction in an unsatisfiable set, but it is not guaranteed to derive an arbitrary true sentence.
- So the proof is turned round using $KB \models \alpha$ iff $KB \wedge \neg \alpha$ is unsatisfiable, the equivalence from §4.2.

How to write one that scores

- **Number every clause** and, beside each resolvent, write *which two clauses* it came from and *which substitution* was used. Examiners award those annotations.
- **Resolve towards the goal.** Start from the negated goal clause and work down: it prunes the search and it reads as a proof rather than a search log.
- **One complementary pair per step.** Cancelling two pairs in one line is unsound and is a standard trap.
- **Standardise variables apart** before each step (§4.4, step 8).
- Finish with the sentence “*the empty clause is derived, so the assumption contradicts the premises, hence the goal is proved*”. It is worth a mark and takes one line.



The shape the answer must take: clauses down the left, each resolvent to its right, the empty clause $\square$ at the foot. Premises are the Colonel West set, asked in 7 papers.

33 papers, but only 26 different premise sets, and four families cover 22 of them

- Likes-all-food / peanuts — 7 papers, five wordings of one puzzle (John/Bill/Sue, Ram/Krishna/Radha, Ram/Anil, Sita/mushroom, Bhogendra/chicken). **Learn this one first.**
- Charlie is a horse / a mammal — 6 papers, three wordings. Tests *inverse relations*, and **two of its six premises are never used.**
- Selling weapons to a hostile nation — 7 papers under six names, one asking for forward chaining instead. Tests *existential instantiation*.
- Light sleeper, hounds and mice — 2 papers. The hardest of the four: the goal is itself an implication, so **negating it gives three clauses.**
- Then 10 one-off sets, from Marcus and Caesar to Curiosity and the cat.

Contradiction Vs extraction (the textbook's two methods)

- **Answer by contradiction:** negate the goal, derive $\square$. Use this whenever the paper says *prove that ...*, which is 21 of the 23 sets.
- **Answer by extraction:** do not negate; chain the clauses forward until the goal literal itself is produced. Use it when the paper asks *who* or *what* rather than *prove*, Eg 72 Ma's "did Marcus hate Caesar?"
- **Same clauses, same resolution rule;** only the starting point differs.

The two things that go wrong in an exam

- **Negating a goal that is an implication.** "If John is a light sleeper then John has no mice" negates to "John IS a light sleeper AND John HAS a mouse", which after skolemization is **three** clauses: $LS(John)$, $Have(John, b)$, $Mouse(b)$. Students negate it to one clause and the proof cannot close.
- **"Has some" is existential.** "Nono has some missiles" is $\exists x Missile(x) \wedge Owns(Nono, x)$, which skolemizes to a **new constant** M_1 : two clauses, $Missile(M_1)$ and $Owns(Nono, M_1)$. Writing $Missile(x)$ with a free variable claims *everything* is a missile.
- Both are worked step by step in the companion, which prints each paper's premises exactly as the paper words them.

4.6 Rule-based reasoning: forward and backward chaining

Differentiate between forward and backward chaining with a suitable example 2 3 4 7 8

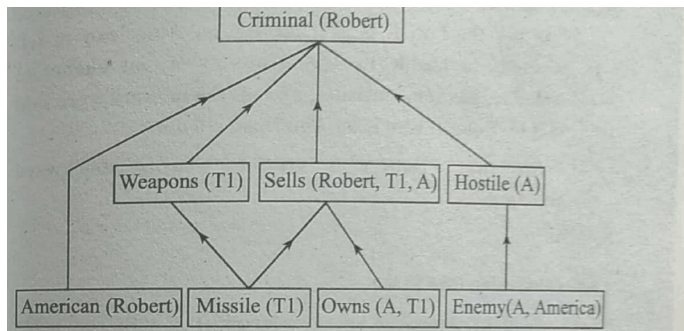
TOP 11/41 70 Ma 4+4, 74 Bh 4, 79 Bh 7, 81 Bh 7, 82 Ka 2+4, 81 Ash 2, 73 Bh 3, 80 Ch 4, 72 Ash 2+5, 78 Po 8, 77 Ch 8 · **always wants an example, not just the table** · in 81 Ash the example is the Colonel West proof, done forward

Forward chaining: data-driven

- Start from the **known facts**. Fire every rule whose premises are all satisfied, add its conclusion to the fact base, repeat until the goal appears or nothing new can be derived.
- **Bottom-up**, and it uses **modus ponens** on definite clauses.
- ✓ Derives **everything** that follows, so it suits monitoring, alarms and planning, where new data arrives and consequences must be found.
- × Does a lot of **irrelevant work**: it proves facts nobody asked for.

Backward chaining: goal-driven

- Start from the **goal**. Find a rule that concludes it, make that rule's premises the new sub-goals, and recurse until every sub-goal is a known fact.
- **Top-down, depth first**, and again modus ponens.
- ✓ Touches **only what the goal needs**, so it suits diagnosis, query answering and theorem proving. **Prolog** works this way.
- × Can loop on recursive rules, and it answers only the question asked.



Forward chaining on the Colonel West premises: facts on the bottom row, one new row per iteration, until *Criminal(Robert)* appears.

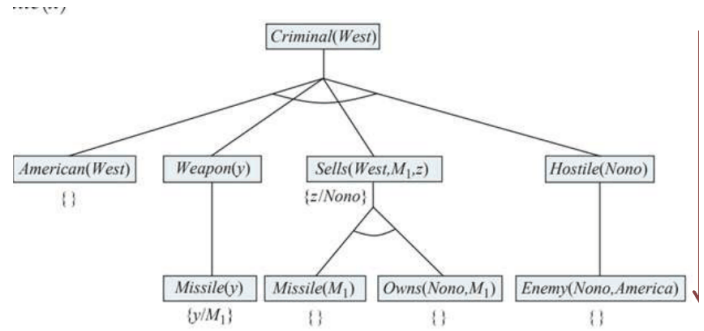


Figure 9.7 Proof tree constructed by backward chaining to prove that West is a criminal. The tree should be read depth first, left to right. To prove *Criminal(West)*, we have to prove the four conjuncts below it. Some of these are in the knowledge base, and others require further backward chaining. Bindings for each successful unification are shown next to the corresponding subgoal. Note that once one subgoal in a conjunction succeeds, its substitution is applied to subsequent subgoals. Thus, by the time FOL-BC-ASK gets to the last conjunct, originally *Hostile(z)*, *z* is already bound to *Nono*.

Backward chaining, same premises: the goal at the top, its four conjuncts below it, each expanded until it meets a fact. The binding that made each unification succeed sits under its node.

	Forward chaining	Backward chaining
Starts from	Known facts / data	The goal to be proved
Direction	Bottom-up, data-driven	Top-down, goal-driven
Question answered	"What can I conclude?"	"Can I prove this?"
Search	Breadth-first in effect: all consequences of the current facts	Depth-first: one sub-goal chain at a time
Work done	Derives many facts, most irrelevant to any one goal	Derives only what the goal needs
Suits	Monitoring, alarms, planning, real-time control, design	Diagnosis, query answering, theorem proving, Prolog
Number of goals	Handles many goals at once	Handles one goal at a time
Efficiency	Slow when the goal is specific	Slow when the fact base is small and rules are deep

- **The one-line discriminator to write:** forward chaining asks *what follows from what I know*, backward chaining asks *what would I need to know for this to be true*. Same rules, same modus ponens, opposite ends.

What is rule-based reasoning? 2 7

HOT 4/41 72 Ma 7, 70 Ma 4+4, 71 Bh 7, 77 Ch 2 · asked as the **2-mark opener** to a backward-chaining question in three of the four

Definition and parts

- Knowledge stored as **IF condition THEN action** rules, and reasoning done by an **inference engine** that decides which rule to fire.
- Three components:
 - **Rule base** – the IF-THEN rules (long-term knowledge);
 - **Working memory** – the facts known right now;
 - **Inference engine** – match, resolve conflicts, fire.
- The engine's cycle is **match** → **conflict resolution** → **execute**, exactly the production system of Ch 2. **Say so:** the examiner set that question too.

Why it is used, and the worked shape

- ✓ Rules are **modular**: add or remove one without touching the rest. ✓ They are readable by a domain expert who is not a programmer. ✓ The chain of fired rules is the explanation, which is how an expert system justifies itself (Ch 7).
 - × No learning, and × large rule bases become slow and can conflict.
 - **The example to give**, since every paper asks for one:
 - R1: IF *fever* AND *cough* THEN *flu*. R2: IF *flu* THEN *prescribe rest*.
 - Facts: *fever, cough*. Forward: R1 fires, *flu* added, R2 fires, *prescribe rest*. Backward from *prescribe rest*: needs *flu*, which needs *fever* and *cough*, both known.
- Two directions, one rule base.**

4.7 Statistical reasoning: uncertainty, Bayes and belief networks

How is reasoning done under uncertainty? What are the approaches for measuring it? 2 3 3+6

80 Ash 3, 81 Ash 2, 79 Ash 3+6 · **always the opener to a Bayes numerical**, so keep it to half a page and spend the time on the calculation

Why logic is not enough

- FOPL is **monotonic and categorical**: a fact is true or false, and adding knowledge never retracts a conclusion. The real world is neither.
- Three sources of uncertainty, and papers want them named:
 - incomplete** knowledge (not all facts are available);
 - imprecise / unreliable** data (a sensor, a symptom);
 - rules that are only usually true** (*a stiff neck suggests meningitis*).
- Writing *stiff neck* → *meningitis* in logic is simply false, and listing every exception is impossible. **So attach a number to the belief instead.**

The approaches for measuring uncertainty

- Probability theory** – degree of belief in $[0, 1]$, updated by **Bayes' rule**. The mainstream answer, and the only one the numerals use.
- Certainty factors** – an ad-hoc $[-1, +1]$ score attached to each rule, as used in MYCIN. Simple, but not probabilistically sound.
- Dempster-Shafer theory** – assigns belief to *sets* of hypotheses, so it can represent *ignorance* separately from *disbelief*.
- Fuzzy logic** – handles *vagueness* (“tall”, “warm”), which is a different thing from uncertainty. Covered in Ch 6.
- Non-monotonic / default reasoning** – assume the normal case, retract the conclusion when contradicted.
- The distinction worth one mark**: probability is about *not knowing which case holds*; fuzzy logic is about *the case not having a sharp boundary*.

What is prior and posterior probability? Explain Bayes' theorem and its role in reasoning 2 3 2+6 3+5

TOP 15/41 71 Ma 3+5, 73 Bh 7, 73 Ma 8, 73 Bh 8, 72 Ash 8, 78 Ch 3+5, 80 Ch 3+5, 80 Bh 2+6, 80 Ba 5+3, 81 Ba 2+6, 80 Ash 3+5, 81 Ash 2+6, 78 Ba 2+6, 82 Ba 2+6, 82 Bh 7 · **15 papers carry a Bayes calculation**, every one of them worked in the companion · the theory part is never more than 3 marks

The vocabulary

- Prior** $P(A)$ – belief in *A* **before** any evidence. Eg 1 person in 50,000 has meningitis.
- Posterior** $P(A|B)$ – belief in *A* **after** evidence *B* is observed. Eg the chance of meningitis *given* a stiff neck.
- Likelihood** $P(B|A)$ – how probable the evidence is if the hypothesis holds. **This is the number a doctor actually knows**, and it is the reason the theorem is needed.
- Marginal** $P(B)$ – the probability of the evidence however it arose.

The theorem

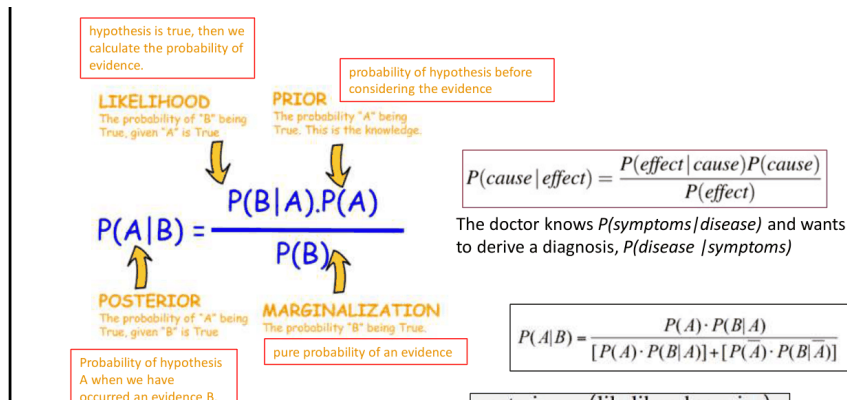
$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- When $P(B)$ is not given, expand it by the **law of total probability**, which is what most papers require:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B|A)P(A) + P(B|\neg A)P(\neg A)}$$

How it aids inference (the marks in the theory half)

- It **reverses the direction of a conditional**. Causal knowledge runs *cause* → *effect* ($P(\text{symptom} | \text{disease})$, stable and measurable); diagnosis needs *effect* → *cause*. Bayes converts one into the other.
- It gives a **principled update rule**: $\text{posterior} \propto \text{likelihood} \times \text{prior}$. Evidence does not replace belief, it revises it.
- It makes the **base rate** explicit. A 99 %-accurate test for a 1-in-10,000 disease still leaves you far more likely to be healthy than ill, because the prior is tiny. **This is asked directly** (72 Ash) and it is the single most useful sentence in the section.
- It composes: a chain of evidence can be applied one piece at a time, which is exactly what a belief network does below.



Getting the arithmetic right

- Name the events **before** substituting. Half the lost marks are a $P(A|B)$ written where $P(B|A)$ was meant.
- If the question gives percentages of a population that split into groups (factories, men and women), the denominator is **always the total-probability sum** over those groups.
- $P(\neg A) = 1 - P(A)$ is almost always needed and almost never given.

What is a causal / belief network? How is reasoning done in it? 2+5 2+6 6 7

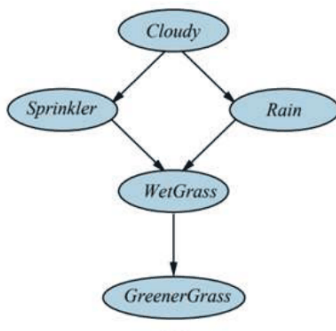
HOT 7/41 71 Ma 2+5, 79 Ch 2+6, 79 Ash 3+6, 77 Ch 6, 73 Bh 7, 80 Ba 5+3, 82 Bh 7 · causal net, belief network, Bayesian network and Bayesian belief network are **four names for one thing** in these papers · two papers print a network and ask for a joint probability (78 Ba, 82 Bh)

Definition and the two parts

- A **directed acyclic graph** in which
 - each **node** is a random variable, and
 - each **edge** $X \rightarrow Y$ says X has a **direct causal influence** on Y ; plus a **conditional probability table (CPT)** at every node giving $P(\text{node} | \text{its parents})$.
- A root node has no parents, so its CPT is just its prior.
- **The whole network in one equation**, and this is what the numerals use:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents}(x_i))$$

- **Why it is worth having:** a full joint distribution over n binary variables needs $2^n - 1$ numbers. The alarm network below has 5 variables and needs **only 10**, because each node depends on its parents alone. That compactness is the answer to *when do we use a Bayesian network*.
- **Conditional independence** is the property being exploited: a node is independent of its non-descendants given its parents.



A causal net: Cloudy influences Sprinkler and Rain, both influence WetGrass. The shape 78 Ba prints.

Reasoning in the network (the 5 or 6 marks)

- **Causal / top-down** – from cause to effect. *Burglary happened; how likely is John to call?* Uses the CPTs directly.
- **Diagnostic / bottom-up** – from effect to cause. *John called; how likely is a burglary?* This is where **Bayes' theorem enters**, and it is the answer to "how is Bayes' theorem used in a belief network" (77 Ch).
- **Intercausal** – between two causes of one observed effect. *The alarm rang and an earthquake is reported, so burglary becomes less likely: explaining away.* Worth naming; almost nobody does.
- **Mixed** – evidence above and below the query node at once.
- **The method for a numerical:** write the product form, substitute one CPT entry per variable, and read $P(\neg X) = 1 - P(X)$ off the same table.

✓ **Variables:** Burglar, Earthquake, Alarm, JohnCalls, MaryCalls

Bayesian Network for Alarm Domain

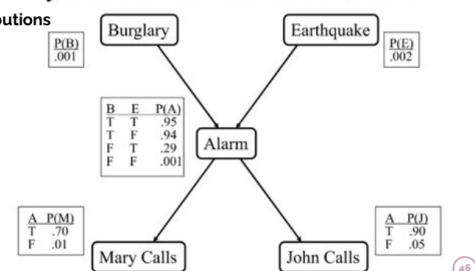
Retrieving Probabilities from the Conditional Distributions

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{Parents}(x_i))$$

• E.g.

$$\begin{aligned} &P(J \& M \& A \& \neg B \& \neg E) \\ &= P(J|A)P(M|A)P(A|\neg B, \neg E)P(\neg B)P(\neg E) \\ &= 0.9 \times 0.7 \times 0.001 \times 0.999 \times 0.998 \\ &= 0.00062 \end{aligned}$$

© Baikuntha Acharya (baikuntha2@gmail.com)



The standard alarm network, with a CPT at every node. *Burglary* and *Earthquake* both cause *Alarm*; the alarm makes *John* and *Mary* call. Five variables, and the graph plus these five tables hold the whole joint distribution in **10 numbers** instead of $2^5 - 1 = 31$. The worked line beside it is the product form applied straight down the network, which is exactly what 78 Ba and 82 Bh ask for.

- **The connection the papers ask for in one sentence:** a belief network stores *causal* conditional probabilities, and Bayes' theorem is what lets you run them *backwards* to a diagnosis. Without the network you would need the full joint distribution; without Bayes you could only reason in the direction the arrows point.

Abbreviations used in these notes

Only these are ever shortened. Anything not on this list is written out in full.

Short	Means	Short	Means
AI	artificial intelligence	ANI	artificial narrow intelligence
AGI	artificial general intelligence	ASI	artificial super intelligence
KR	knowledge representation	KRR	knowledge representation and reasoning
PEAS	performance measure, environment, actuators, sensors	GPS	General Problem Solver
NLP	natural language processing	NLU	natural language understanding
NLG	natural language generation	CAPTCHA	completely automated public Turing test to tell computers and humans apart
ML	machine learning	ANN, NN	artificial neural network
MLP	multilayer perceptron	BP	back-propagation
SOM	self-organizing map	GA	genetic algorithm
FIS	fuzzy inference system	ID3	Iterative Dichotomiser 3
CSP	constraint satisfaction problem	DFS	depth first search
BFS	breadth first search	DLS	depth limited search
IDS	iterative deepening search	UCS	uniform cost search
CNF	conjunctive normal form	WFF	well-formed formula
FOL	first-order logic	FOPL	first-order predicate logic
RRS	resolution refutation system	BBN	Bayesian belief network

Short	Means	Short	Means
ES	expert system	KB	knowledge base
CD	conceptual dependency	LISP	list processing (the language)
MOP	memory organization packet	isa	the class-to-class link of a semantic net (<i>instance</i> is the individual-to-class one)
attr(s)	attribute(s)	repo	repository
freq	frequency	config	configuration
info	information	no.	number
Eg	example	Vs	compare / differentiate
w/, w/o	with, without	+Fig	draw the figure
→	leads to, therefore	=	is, equals

Reading the year tags and chips

Exam-paper month codes: Ba = Baishakh · Jth = Jestha · Shr = Shrawan · Bh = Bhadra · Ash = Ashwin · Ka = Kartik · Po = Poush · Ma = Magh · Ch = Chaitra. Years are Bikram Sambat, so 80 Ch is 2080 Chaitra.

Bold year tags are Regular exams, plain are Back exams. A header printing "Regular / Back" in one box counts as Regular.

Three course codes, three faces. Plain = CT 653 (BCT, III-II, 26 papers) · monospace = CT 710 (BEI, IV-I, 7 papers) · *italic* = CT 78506 (BEX, IV-II, 8 papers). **The same year code names different papers under different codes:** 77 Ch and 77 Ch are two different exams, and only the font tells them apart.

Tier chips over all 41 papers, 2069–2082 BS: **TOP /41** = topic asked in **8 or more** papers · **HOT /41** = **4 to 7** papers · **PIN /41** = **1 to 3** papers but worth ≥ 6 marks.

Mark chips such as **2+6** are the marks that question actually carried. **syllabus, no PYQ yet** marks a syllabus topic never yet examined but written up anyway. **verified/added** marks anything researched and added that no source note in this folder covers.